

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа № 4

Технический дизайн микросервисной архитектуры

Выполнил:

Колесников Игорь Евгеньевич

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное API Jobby на сервисы с отдельными базами данных, определить синхронные взаимодействия и описать API и ошибки в OpenAPI.

Вариант проекта: Jobby - сервис поиска работы.

Ход работы

Анализ монолита и границы сервисов

Монолит из labs/lab1 содержит регистрацию и профили, компании и вакансии, резюме, отклики и справочники. Разделение выполнено по функциональным областям. Каждый сервис владеет своими таблицами и не обращается напрямую к БД другого сервиса.

Сервис	Порт хоста	База данных	Собственные таблицы
user-service	3101	jobby_user	users, job_seekers, employers
vacancy-service	3102	jobby_vacancy	companies, vacancies, vacancy_skills
resume-service	3103	jobby_resume	resumes, work_experiences, educations, resume_skills
application-service	3104	jobby_app	applications
dictionary-service	3105	jobby_dict	countries, cities, industries, skills, employment_types, degree_types
notification-service	нет HTTP-порта	нет	Получает события из RabbitMQ и записывает их в журнал контейнера

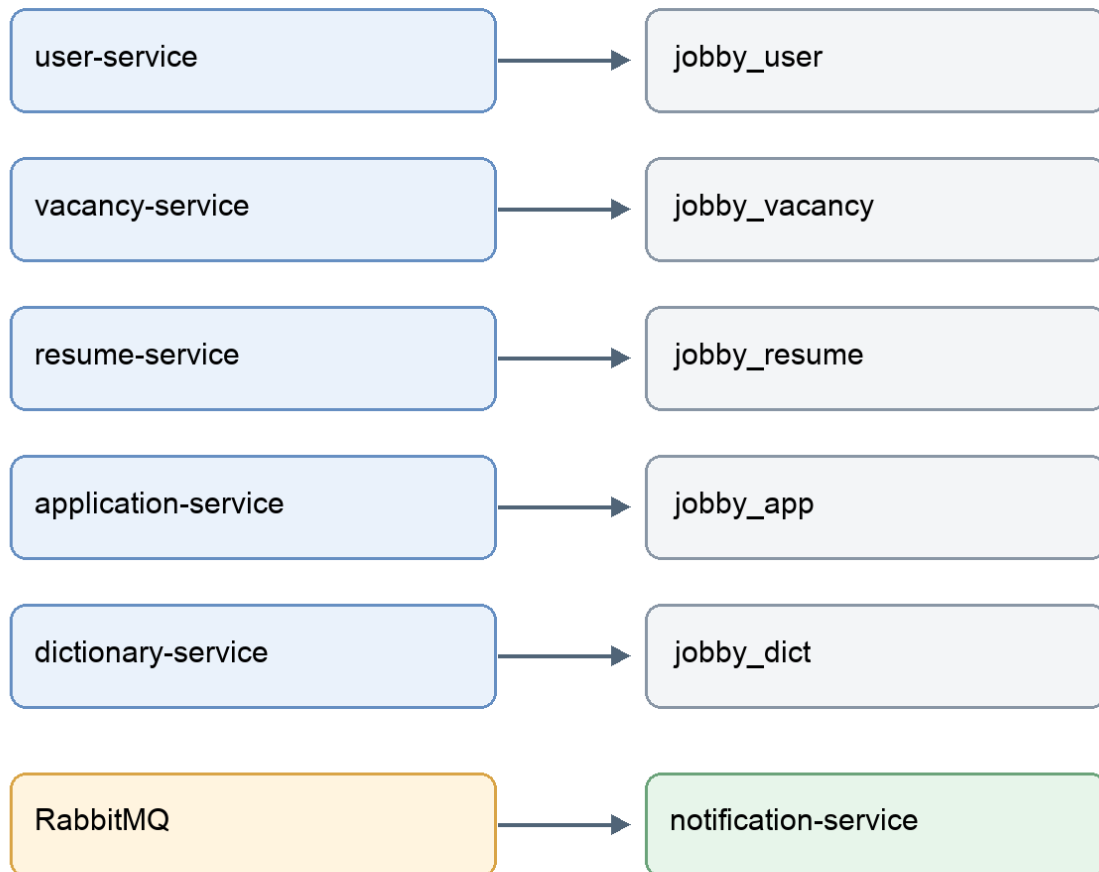
Внешние идентификаторы (employer_id, job_seeker_id, resume_id, vacancy_id, skill_id) хранятся как UUID. В БД другого сервиса внешние

ключи для них не создаются: существование и право доступа проверяются через HTTP.

Схема взаимодействия

Архитектура Jobby

Пять HTTP-сервисов владеют пятью отдельными БД



application-service публикует события откликов в RabbitMQ

Рисунок 1 - Схема микросервисной архитектуры Jobby

В текущем docker-compose.yml клиент обращается к сервисам напрямую по указанным портам. Отдельный API Gateway не реализован.

Синхронные вызовы

Внутренние маршруты начинаются с /internal/. Сервисы передают заголовок X-Internal-Secret, значение которого задаётся переменной INTERNAL_SECRET. Публичные маршруты используют JWT из cookie access_token и проверяют роль пользователя.

Инициатор	Получатель	Пример внутреннего маршрута	Назначение
vacancy-service	user-service	GET /internal/employers/by-user/{userId}	Проверка работодателя и владельца вакансии
vacancy-service	user-service	PATCH /internal/employers/by-user/{userId}/company	Привязка созданной компании к работодателю
vacancy-service	dictionary-service	GET /internal/skills/{skillId}	Проверка навыка
resume-service	user-service	GET /internal/seekers/by-user/{userId}	Проверка соискателя
resume-service	dictionary-service	GET /internal/skills/{skillId}	Проверка навыка резюме

Инициатор	Получатель	Пример внутреннего маршрута	Назначение
application-service	resume-service	GET /internal/resumes/{resumeId}	Проверка резюме и владельца
application-service	vacancy-service	GET /internal/vacancies/{vacancyId}	Проверка вакансии
application-service	user-service	GET /internal/employers/by-user/{userId}	Проверка владельца отклика
vacancy-service	application-service	GET /internal/vacancies/{vacancyId}/applications	Получение откликов для автора вакансии

Полный перечень публичных и внутренних операций, параметры, схемы JSON и ожидаемые ответы приведены в `openapi.yaml`. Основные ошибки: 401 - нет действующей сессии, 403 - недостаточно прав, 404 - объект не найден, 409 - повторное создание объекта или отклика, 422 - неверные данные запроса, 502 - недоступен вызываемый сервис.

Порядок перехода от монолита

1. Выделить перечисленные таблицы в пять баз PostgreSQL и оставить владельцем каждой таблицы один сервис.
2. Вынести регистрацию и профили в user-service, справочники - в dictionary-service.
3. Перенести компании и вакансии в vacancy-service, резюме - в resume-service, отклики - в application-service.
4. Заменить обращения через ORM к чужим таблицам внутренними HTTP-вызовами; проверять владельца данных на стороне сервиса, обрабатывающего публичный запрос.
5. Подключить RabbitMQ для уведомлений об откликах (см. отчёт Д35).
6. Собрать сервисы через labs/lab23/docker-compose.yml и проверить основной сценарий: регистрация → вакансии → резюме → отклик → просмотр и смена статуса.

Вывод

Проект разделён на шесть сервисов, включая consumer уведомлений, с пятью независимыми базами. Синхронные вызовы выполняются по HTTP, а события отклика передаются через RabbitMQ. Спецификация API находится рядом с отчётом.